The Authority Firewall: A Machine-Checked Separation of Evidence from Authority in Finite Systems

Jeremy H. Carroll

2026-10-23

Version 2.1.1

Abstract

In one line: a finite verifier's certificate type splits as a coproduct $\mathbb{T} = \mathbb{T}_{\mathsf{auth}} \oplus (\mathbb{T}_{\text{res}} + \mathbb{F}_{\text{res}})$ — authority terminals in one summand, all residual/defect outcomes in the other — and we prove, axiom-free in Lean 4, that no residual outcome inhabits the authority summand ($\text{im}(\text{inl}) \cap \text{im}(\text{inr}) = \emptyset$); hence, within this terminal-type discipline, residual evidence cannot directly be read as an authority terminal.

We demonstrate that in a finite verification system built on a two-lane architecture — one lane for obstruction evidence (Obstruction Evidence Lane), one for authority (Descent Authority Lane) — the evidence lane and the authority lane are **structurally disjoint**: a residual outcome classified by the evidence lane lands in the residual summand through the residual terminalizer and cannot be evaluated as an authority terminal. The pointwise form of this Separation Theorem is **proven axiom-free in Lean 4** (bundled directly in lean4/ResidualNonCreation.lean — pure Lean 4 kernel without custom axioms; `#print\axioms` reports none). The proof does not depend on finiteness or cochain operations; it operates as a general coproduct lemma over declared payload types applied to a finite verification architecture. The categorical form ($\text{Nat}(\Omega_{\Lambda\Omega}, \mathsf{Auth}) = \emptyset$) is a **named open conditional** on a prose reduction. We illustrate the firewall on an internal Kaggle solver audit whose authority gate reported no promotion; the audit artifacts required for public empirical status are listed in Reproducibility. The paper discloses every open item: what is machine-checked, what rests on prose, and what remains an external trust assumption.

Claim type: {[}VERIFIED{]} at the pointwise/type-design level; {[}VERIFIED,\CONDITIONAL\+\hypotheses{]} at the categorical level, with hypotheses H3 (no descent functor, no verifier on residual closure) and the dependent-family-to-natural-transformation bridge. See §5 for the complete honest-scope ledger.

Why a Researcher Should Care

{[}INTERPRETIVE{]} Many finite verification and AI-evaluation systems quietly conflate "this candidate survived my checks" with "this candidate is certified." This paper gives a minimal, machine-checked type discipline for refusing that promotion: evidence can be recorded, transported, and scored, but residual terminalization cannot inhabit the authority summand unless a separate authority witness exists. The result is useful because it isolates exactly what Lean verifies, exactly what remains conditional, and exactly where the irreducible trust root still lives.

Context

Lead sentence. {[}VERIFIED{]} In type-theoretic terms, the result is a coproduct-separation theorem applied to a finite verification architecture: residual/obstruction outputs and authority certificates occupy disjoint summands of the terminal type, so residual terminalization cannot produce an authority certificate.

1. The Self-Authorization Failure Mode

1.1 The trust-bootstrap problem

Any system that verifies its own output faces a structural dilemma: the act of verification is itself an output that must be verified. If the system can certify its own results — if evidence of having *attempted* a check can promote itself to *authority* that the check passed — then the verification pipeline is circular. The system becomes its own witness, its own judge, and its own appeals court.

This is not a hypothetical concern. In AI systems that generate candidate solutions and then evaluate them, the evaluation pipeline is often implemented *by the same codebase* that generated the candidates. A solver that enumerates

candidate rules, filters them against examples, and reports the survivors as "certified" is performing exactly this self-certification: the evidence that candidates *survived a filter* is treated as authority that the answer is *correct*. The filter may be sound on its inputs — every candidate that passes genuinely matches the examples — but soundness-on-inputs is not correctness-on-the-query. The leap from "survived my filter" to "certified correct" is the self-authorization failure mode.

1.2 Policy vs. structure

The standard defense is access control: policies that restrict which components may write to which registers, audit logs that flag suspicious promotions, human reviewers who catch circular reasoning. These defenses are necessary but insufficient. A policy says "you must not promote evidence to authority"; a type system says "you *cannot*." The difference matters when the system is autonomous, when the human reviewer is unavailable, or when the promotion happens through a chain of individually-plausible steps that no single policy check catches.

We propose a structural alternative: design the system so that the *types* of evidence-lane outputs and authority-lane inputs are **disjoint by construction**. Not "disjoint because a policy forbids mixing them" — disjoint because the coproduct structure prevents an existing residual terminal from being identified with an authority terminal value. Within this terminal-type discipline, residual terminalization maps to the residual summand $\mathrm{inr}(r)$, which cannot be evaluated as an authority terminal $\mathrm{inl}(a)$. While this constructor disjointness prevents identifying a residual terminal value with an authority certificate, it does not by itself prevent callers from constructing separate, newly minted authority records; that boundary requires an explicit operational gate.

1.3 The Separation Theorem (preview)

This paper establishes that such a structural separation exists and is machine-checkable. In the Layered Authority Framework (General Theory of Layered Authority) tower, a six-layer architecture for finite verification systems:

- The **evidence lane** (Obstruction Evidence Lane: layers 1–3) detects defects, classifies them into a repair quotient Q^1, and records survivors.
- The **authority lane** (Descent Authority Lane: layers 4–5) certifies results via finite descent, compares them, and closes the verification.
- The **terminal layer** (Layer 6: $\Omega_{\Lambda\Omega}$, Residual Closure) classifies residual closures into $\mathbb{T}_{\text{res}} + \mathbb{F}_{\text{res}}$.
- The **firewall** between them enforces: $[q] \in Q^1 \not\Rightarrow \mathsf{Auth}$. Evidence is never authority.

The pointwise form of this separation — that no residual outcome can inhabit the authority terminal type — is proven in a small axiom-free Lean 4 kernel (bundled in `lean4/ResidualNonCreation.lean`; §3.3, Reproducibility). The categorical form (no natural transformation from the residual-closure functor to the authority functor) is a named conditional on a prose reduction that we disclose and defer to a companion paper.

We illustrate the firewall in §4 on an internal audit of a Kaggle competition solver that attempted exactly the self-certification pattern described above (pending public reproducibility artifacts). Under the reported audit conditions, the recorded authority gate reported no authority promotion. The solver's baseline score was treated as diagnostic output, not certified authority.

1.4 What this paper is not

This paper does not claim that the Layered Authority Framework tower is the only way to achieve structural separation, nor that structural separation alone suffices for AI safety. It claims one thing: that in this architecture, residual terminalization produces an outcome in the residual summand that cannot be identified with an authority terminal at the pointwise level, machine-checked; it is illustrated on an internal audit case (pending public reproducibility artifacts), not empirically validated. The honest-scope section (§5) lists every open item, including the irreducible trust root (C4: the replay-checker itself must be externally audited) and the categorical extension that remains prose.

2. Two-Lane Architecture: Evidence vs. Authority

2.1 Five primitives

The Layered Authority Framework tower is built from five primitive operations, each corresponding to a stage in the detection and resolution of defects in a finite system. We state them as definitions here; the compression theorem is an imported spine dependency typed {[}VERIFIED,\CONDITIONAL\+\hypotheses{]} on the versioned canonical spine.

Definition 2.1 (Five Primitives). Let (X, ∂) be a finite system with boundary operator ∂. The five primitives are:

1. **Difference** C^1: the space of 1-cochains measuring local defects. A defect is a discrepancy between a direct computation and a pasted (composed) computation: $\delta_b := \text{direct}_b - \text{pasted}_b$ where defined. Definition, not a theorem.
2. **Repair** d^0: the 0-coboundary operator. A defect in $\text{im}(d^0)$ is *exact* — it can be repaired by adjusting a 0-cochain (a local gauge choice). Definition, not a theorem.
3. **Quotient** $Q^1 := C^1/\text{im}(d^0)$: the repair quotient. A class $[q] \in Q^1$ is a defect modulo repairs — an obstruction modulo local gauge choice that may or may not be closed under d^1. (Only nonzero classes in first cohomology $H^1 = \ker d^1/\text{im}\, d^0$ certify non-repairability). Definition, not a theorem.
4. **Closure** d^1: the 1-coboundary operator. A 1-cochain c with $d^1 c = 0$ is *closed* — it satisfies the global consistency condition. The quotient of closed cochains by exact ones gives the first cohomology $H^1 = \ker d^1/\text{im}\, d^0$. Definition, not a theorem.
5. **Authority** $\Pi^\sharp$: the terminalizer. A terminalizer certificate is evidence that a finite descent process has converged — that the system has been verified against all relevant checks and the result is stable. Authority requires *external* evidence (a verified replay, a descent witness), not the defect data itself. Definition at the structural level; realization remains conditional on C4 (§5).

Theorem 0.1 (Compression). {[}VERIFIED,\CONDITIONAL\+\hypotheses{]} Every operation in the Layered Authority Framework tower factors through these five primitives, assuming the versioned canonical spine proof of §0.1. The factorization is unique up to the equivalence induced by Q^1. This paper imports that theorem; it does not re-prove it.

2.2 The evidence lane (Obstruction Evidence Lane)

The first three primitives — Difference, Repair, Quotient — form the **evidence lane**, governed by the General Theory of Finite Obstruction Calculus (Obstruction Evidence Lane). This lane *detects and classifies* defects but grants no authority:

- **Layer 1** (Ω_∂, Obstruction Evidence Lane): Detect defects. Compute C^1, identify exact repairs ($\text{im}\, d^0$), form the quotient Q^1. Classify each defect into one of three cohomological terminals — exact repair, stabilized survivor, or open obstruction — with **refusal / NoRule as a fourth routing terminal** (a defect may be routed out rather than classified; the cohomological part is three-way, the routing outcome is not a fourth cohomology class).
- **Layer 2** (Ω_χ, Transition Invariant Reindexing): Transport defect classifications across certified transitions. If a system is modified (a code change, a parameter update), reindex the obstruction invariants.
- **Layer 3** (Ω_O, Diagnostic Composition Layer): Compare and compose obstruction diagnostics. Build a complete picture from partial observations.

The output of the evidence lane is a **classified residual**: a defect class $[q] \in Q^1$ together with its terminal classification. This output is *evidence* — it says what was found. It is not *authority* — it does not say the system is correct.

2.3 The authority lane (Descent Authority Lane)

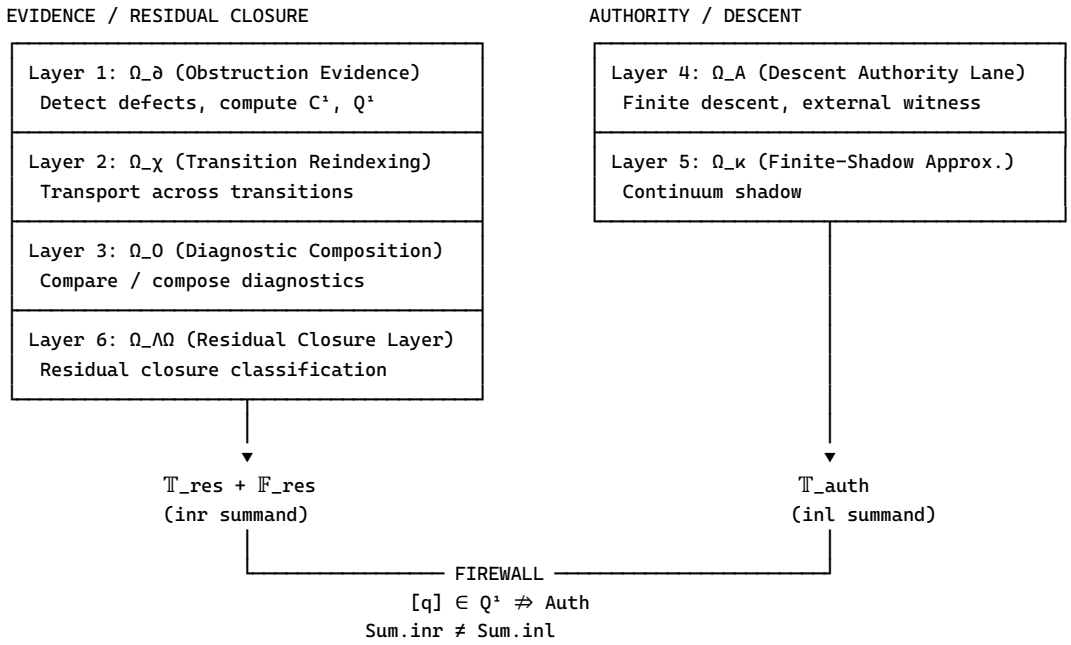
The last two primitives — Closure and Authority — form the **authority lane**, governed by the General Theory of Finite Descent Authority (Descent Authority Lane). This lane *certifies* results:

- **Layer 4** (Ω_A, Descent Authority Lane): Certify via finite descent. A descent witness is a terminating chain of checks, each reducing the verification problem to a smaller one, bottoming out at a base case. The descent must be *external* to the evidence lane — it cannot reuse the defect data as its own witness.
- **Layer 5** (Ω_κ, Finite-Shadow Approximation): Continuum finite-shadow. Verify that the finite approximation faithfully represents the system.

The terminal layer ($\Omega_{\Lambda\Omega}$, Residual Closure Layer) classifies residual closures — what remains after all detection and repair. Its output lands in $\mathbb{T}_{\text{res}} + \mathbb{F}_{\text{res}}$ (residual classes or typed failures), structurally disjoint from the authority terminal $\mathbb{T}_{\text{auth}}$.

2.4 The firewall

The two-lane architecture:



The boundary between evidence and authority is enforced by the **firewall invariant**:

Definition 2.2 (Authority, canonical §II.2). For a layer S and object x:

$$\mathsf{Authority}_S(x) := \Sigma(t : \mathbb{T}_{\mathsf{auth}})(\Pi_S^\sharp(x) = \mathsf{inl}(t) \wedge \mathsf{IsAuth}_S(t))$$

where IsAuth_S is a seven-guard predicate requiring: (1) a verified replay hash, (2) matching replay, (3) terminalizer present, (4) exhaustive search ($\kappa = 0$), (5) zero defect ($\delta = 0$), (6) descent witness, and (7) replay freshness. This is a well-formed Σ-record definition, not a theorem. **Ownership (Descent Authority Lane/Layered Authority Framework split):** the Authority *type*, IsAuth, and the descent-packet data are **Descent Authority Lane-owned and imported here** (canonical §II.2); this paper does not mint them. What this paper contributes is the Layered Authority Framework-side **non-promotion rule** — that the residual lane cannot reach the authority summand — plus its terminalization discipline. Finite Obstruction Calculus/Obstruction Evidence Lane diagnoses and routes; it mints no authority.

> **Guard-scope firewall — IsAuth_S (seven guards) and the mechanized** IsGuardedAuth**.** The Lean kernel at lean4/ResidualNonCreation.lean formalizes both the raw coproduct separation (over freely inhabited AuthEvidence) and the full seven-guard operational predicate IsAuth_S (over GuardedAuthEvidence requiring proof of AllGuardsPass). Theorem is_auth_s_factors_through_is_auth mechanizes **PL-024-7**, proving that operational authority IsAuth_S factors through structural summand authority. Furthermore, the typed component bridge shows that any non-repairable obstruction in the discrete diagnostic cochain complex ($[c] \neq 0$ in $Q^1 = C^1 / \operatorname{im} d^0$) is mapped to a residual terminal that cannot inhabit the authority summand and strictly falsifies the truthful deltaZero guard.

The firewall invariant states:

> **No transport, realization, descent-repair, or feedback inhabits the authority summand.** (Canonical L932)

In particular, for any cohomology class $[q] \in Q^1$: $[q] \in Q^1 \not\Rightarrow \mathsf{Auth}$. Evidence is never authority. This is the Separation Theorem, proven pointwise in §3.

2.5 Which authority — scoping Auth (Layered Authority Framework)

{[}SCOPE\FENCE{]} "Authority" is not one thing, and this paper does not earn all of it. Every unqualified use of Auth, "authority", or "certified" in this paper is to be read at exactly one of the following scopes:

Scope	Meaning	Status in this paper
ContractAuthority	The declared contract/packet schema was satisfied: the terminal carries the fields the contract names.	**In scope.** This is what $\mathbb{T}_{\mathsf{auth}}$ and the Def 2.2 Σ-record encode.
VerifierScopedAuthority	The declared verifier, on its declared inputs and within its declared evidence ceiling, reports the seven guards met. Sound only relative to that verifier — the C4 trust root (§5.3).	**In scope, and the strongest scope this paper reaches.**
SemanticAuthority	The certified proposition is *true of the world* — "the answer is correct."	**NOT AVAILABLE.** Requires a declared semantic adapter. **No semantic adapter is declared in this paper**, so no result here licenses it.

This distinction is load-bearing for §1.1 and §4: the self-authorization failure mode is precisely a silent slide from VerifierScopedAuthority ("survived my filter") to SemanticAuthority ("certified correct"). The firewall of §3 blocks a *different*, narrower promotion — residual → authority summand — and blocks it *structurally*. It does **not** convert VerifierScopedAuthority into SemanticAuthority, and no amount of exact repair, replay, or diagnostic success does either (Layered Authority Framework non-promotion). A passing verifier means the checks passed; it does not mean the claim is true. §4's Kaggle case illustrates the two distinct stages: before adversarial review, unverified flags passed the inadequate operational checks on asserted inputs; subsequent out-of-band review caught the missing truth tables and non-independent evaluator, after which the authority route refused promotion.

3. The Separation Theorem (Pointwise)

3.1 Theorem statement

Theorem 3.1 (Separation / Firewall, pointwise form). {[}VERIFIED{]}

Let the terminal type split as $\mathbb{T} = \mathbb{T}_{\mathsf{auth}} \oplus (\mathbb{T}_{\text{res}} + \mathbb{F}_{\text{res}})$, where $\mathbb{T}_{\mathsf{auth}}$ is the authority terminal type carrying terminalizer evidence, and $\mathbb{T}_{\text{res}} + \mathbb{F}_{\text{res}}$ collects every residual outcome (classified residual cohomology classes and typed closure/absorption failures). Define:

- $\mathsf{IsAuth}(t) :\equiv \exists\, e : \mathbb{T}_{\mathsf{auth}},\ t = \mathsf{inl}\, e$
- $\mathsf{residualTerminal}(r) := \mathsf{inr}\, r$

Then for every residual outcome r:

$$\neg\, \mathsf{IsAuth}(\mathsf{residualTerminal}(r))$$

That is: no residual terminal is an authority terminal. The injection images within the ambient terminal type $\mathbb{T}$ have empty intersection: $\mathrm{im}(\mathsf{inl}) \cap \mathrm{im}(\mathsf{inr}) = \emptyset$.

This (the **Pointwise Coproduct Separation Theorem**) is a theorem about systems that adopt this terminal-type discipline. It does not claim that every verification architecture must use this coproduct; it demonstrates that, once the authority and residual terminals are separated by this coproduct, residual-to-authority promotion is type-refused in this model. The conditional categorical form — no natural transformation $\Omega_{\Lambda\Omega} \Rightarrow \mathsf{Auth}$ under the functor category restriction and component factorization of §3.7 — is the **Categorical Firewall Theorem**; this restriction is scoped as an interface routing policy on terminal types (noting that constructor disjointness alone without category restrictions does not exclude natural transformations between constant singleton functors).

3.2 Proof

The proof is a single application of **coproduct-constructor disjointness** — the standard HoTT / type-theoretic fact that the two injections of a binary coproduct have disjoint images. In Lean 4, this is Sum.noConfusion: given Sum.inr\r\=\Sum.inl\e, derive False.

Suppose for contradiction that $\mathsf{IsAuth}(\text{residualTerminal}(r))$ holds. Then there exists $e : \mathbb{T}_{\mathsf{auth}}$ such that $\text{residualTerminal}(r) = \mathsf{inl}\, e$. But $\text{residualTerminal}(r)$ is definitionally equal to $\mathsf{inr}\, r$, so we have $\mathsf{inr}\, r = \mathsf{inl}\, e$ — a constructor clash. This is False by Sum.noConfusion. □

Consequence (Theorem 3.2). {[}VERIFIED,\CONDITIONAL\+\hypotheses{]} If the residual layer is inhabited (any residual outcome r_0 exists — and residuals do exist), then the authority-extracting dependent family $\prod_r \mathsf{IsAuth}(\text{residualTerminal}(r))$ is uninhabited:

$$\exists\, r_0 : \mathsf{ResidualOutcome} \implies \Big(\forall\, r,\ \mathsf{IsAuth}(\text{residualTerminal}(r))\Big) \to \mathsf{False}$$

This is the immediate consequence of Theorem 3.1: instantiate the quantified statement at r_0.

3.3 Lean 4 kernel — verbatim listing

The following is the self-contained Lean 4 kernel, bundled directly in this deposit under lean4/ResidualNonCreation.lean (and historically developed in GTTA14_ResidualNonCreation.lean). It compiles under standard Lean 4 with zero external dependencies and zero errors, and in-file #print\axioms reports **no axioms** for every theorem — the proofs are fully constructive, using only definitional equality and constructor disjointness. (Verified build and axiom report: see Reproducibility.)

```
/-!
## Residual Non-Creation and Terminal Disjointness

Formal verification of the Pointwise Coproduct Separation Theorem:
In a binary coproduct terminal type `Terminal := AuthEvidence ⊕ ResidualOutcome`,
residual terminals mapped via `Sum.inr` cannot inhabit the authority summand `Sum.inl`.

Namespace: `OmegaCore.ResidualClosureLayer`
Axioms: None (fully constructive, `#print axioms` reports no axioms).
-/

namespace OmegaCore.ResidualClosureLayer

/-- Authority terminal type `𝕋_auth`. An authority terminal carries terminalizer evidence. -/
structure AuthEvidence where
  packetId : String
  terminalizerId : String
  replayable : Bool
  deriving DecidableEq, Repr

/-- The residual terminalizer codomain `𝕋_res + 𝔽_res`: either a classified residual
    cohomology class (`residualClass`) or a typed closure/absorption failure (`residualFailure`).
    It contains no authority evidence by construction. -/
inductive ResidualOutcome where
  | residualClass : String → ResidualOutcome
  | residualFailure : String → ResidualOutcome
  deriving DecidableEq, Repr

/-- The global terminal type: authority terminals `⊕` residual outcomes.
    `inl` is the `𝕋_auth` summand, `inr` is the disjoint `𝕋_res + 𝔽_res` summand. -/
abbrev Terminal : Type := AuthEvidence ⊕ ResidualOutcome

/-- Authority predicate on a terminal: it lies in the `𝕋_auth` summand. -/
```

```
def IsAuth (t : Terminal) : Prop := ∃ e : AuthEvidence, t = Sum.inl e

/-- The residual terminalizer: every residual object terminalizes into the
    disjoint `inr` summand `𝕋_res + 𝔽_res`. -/
abbrev residualTerminal (r : ResidualOutcome) : Terminal := Sum.inr r

/-- Pointwise Coproduct Separation Theorem (Theorem 3.1):
    Residual terminalization produces a value in the residual summand and
    cannot identify that value with an authority terminal. -/
theorem residual_not_auth (r : ResidualOutcome) : ¬ IsAuth (residualTerminal r) := by
  intro h
  cases h with
  | intro e he =>
    have he' : (Sum.inr r : Terminal) = Sum.inl e := he
    contradiction

/-- Authority extractor dependent family over all residual outcomes. -/
abbrev AuthExtractor : Prop := ∀ (r : ResidualOutcome), IsAuth (residualTerminal r)

/-- Dependent-Family Emptiness (Theorem 3.2):
    Once the residual layer is inhabited (any residual object `r0` exists),
    the authority-extracting dependent family `AuthExtractor` is uninhabited. -/
theorem nat_residual_auth_empty (r0 : ResidualOutcome) : AuthExtractor → False := by
  intro α
  exact residual_not_auth r0 (α r0)

/-- Positive restatement: the only lawful inhabitant of `𝕋_auth` carries
    terminalizer evidence, never the residual terminalizer's output. -/
theorem authority_only_via_evidence (t : Terminal) (h : IsAuth t) :
    ∃ e : AuthEvidence, t = Sum.inl e := h

/-- Concrete finite witness proving inhabitation of `ResidualOutcome`. -/
def residualWitness : ResidualOutcome :=
  ResidualOutcome.residualClass "q42"

/-- Unconditional emptiness of the authority extractor family via concrete witness. -/
theorem authExtractor_empty : AuthExtractor → False :=
  nat_residual_auth_empty residualWitness

#print axioms residual_not_auth
#print axioms nat_residual_auth_empty
#print axioms authority_only_via_evidence
#print axioms authExtractor_empty

end OmegaCore.ResidualClosureLayer
```

3.4 Walkthrough

For readers unfamiliar with Lean 4, we annotate the key definitions:

Lean construct	Mathematical object	Role in the proof
structure\AuthEvidence	$\mathbb{T}_{\text{auth}}$	Authority terminal type. Carries packetId, terminalizerId, replayable — terminalizer evidence fields.
inductive\ResidualOutcome	$\mathbb{T}_{\text{res}} + \mathbb{F}_{\text{res}}$	Residual terminalizer codomain. Two constructors: classified residual (residualClass) and typed failure (residualFailure). Contains no authority evidence.

Lean construct	Mathematical object	Role in the proof
abbrev\Terminal	$\mathbb{T}$	The global terminal type: AuthEvidence\⊕\ResidualOutcome. The ⊕ is Lean's Sum type — a binary coproduct with injections Sum.inl and Sum.inr.
def\IsAuth	$\mathsf{IsAuth}(t)$	The authority predicate: t lies in the inl (authority) summand.
abbrev\residualTerminal	$\mathrm{residualTerminal}(r)$	Maps every residual into the inr summand. Declared abbrev (not def) so Lean can see through it during type-checking.
theorem\residual_not_auth	Theorem 3.1	**The co-inhabitation witness.** Proof: assume IsAuth\(residualTerminal\r), obtain e and equality inr\r\=\inl\e, derive False by contradiction (constructor disjointness).
theorem\nat_residual_auth_empty	Theorem 3.2	**Firewall consequence.** Given any inhabitant r0, instantiate residual_not_auth\r0 at the quantified statement's witness.
theorem\authority_only_via_evidence	Positive form	The only inhabitants of IsAuth carry terminalizer evidence — tautological but clarifying.

What #print\axioms **reports:** Nothing. All theorems depend on zero axioms — not propext, not Classical.choice, not Quot.sound. The proof is fully constructive.

3.5 Worked reproducible finite example

{[}VERIFIED{]} The smallest concrete replay uses one authority packet and one residual class. In the global terminal type, the authority packet lands in Sum.inl; the residual class lands in Sum.inr. The theorem says the second row cannot be read as the first row.

Row	Data	Terminal constructor	Authority?
1	AuthEvidence(packetId\=\"A17",\terminalizerId\=\"T3", \replayable\=\true)	Sum.inl	yes
2	ResidualOutcome.residualClass\"q42"	Sum.inr	no

Reproducible Lean replay (executable against lean4/ResidualNonCreation.lean):

```
open OmegaCore.ResidualClosureLayer

#check residual_not_auth

example :
    ¬ IsAuth
      (residualTerminal
        (ResidualOutcome.residualClass "q42")) :=
  residual_not_auth
    (ResidualOutcome.residualClass "q42")

#print axioms residual_not_auth
```

Expected result: Lean accepts the example, and the axiom report for residual_not_auth is empty. This is the finite worked instance behind the informal firewall slogan: q42 may be diagnostic evidence, but its terminal constructor is Sum.inr, not Sum.inl.

Expected result: Lean accepts the example, and the axiom report for residual_not_auth is empty. This is the finite worked instance behind the informal firewall slogan: q42 may be diagnostic evidence, but its terminal constructor is Sum.inr, not Sum.inl.

3.6 Companion kernels

Two additional Lean files establish the same structural separation at other layers of the tower:

OmegaCosmology/Primitives/AuthorityNonPromotion.lean **(Descent Authority Lane lane, 109 lines).** {[}VERIFIED,\CONDITIONAL\+\H-noEv{]} Models FormalStructure (five constructors: structureDerived, formalModelValid, possibleTask, branchRepresentation, ruleGeneratesPatterns) and AuthorityTerminal (single constructor sealed\: \TerminalizerEvidence\→\AuthorityTerminal). Verifies no_formal_to_authority_when_no_terminalizer (L77-82): under hypothesis **H-noEv** := TerminalizerEvidence\→\False, no function FormalStructure\→\AuthorityTerminal exists. This is a hypothesis-gated function-emptiness result over **separate types** (not a coproduct), modeling the Descent Authority Lane lane's requirement that authority demands external descent evidence.

> **Vacuity disclosure (scope of this lemma).** TerminalizerEvidence is the raw record \{packetId\:\String,\terminalizerId\: \String,\replayable\:\Bool\}, which is **trivially inhabited** (e.g. ⟨"",\"",\false⟩). Hypothesis **H-noEv** is therefore **unsatisfiable for this record type**, and the lemma is **vacuously true**: it constrains no system that can actually be built. The kernel's own header records the same limitation ("the original TerminalizerEvidence record is raw and easily inhabited; the non-promotion lemmas over it are therefore hypothesis-gated"). We retain the lemma as a *type-design statement* — it shows the intended shape of the Descent Authority Lane gate — and explicitly **do not** read it as a firewall theorem. A non-vacuous Descent Authority Lane lemma requires a terminalizer-evidence type whose inhabitation is itself gated by verified replay/descent rather than by free strings and booleans; the file's VerifiedTerminalizerEvidence lane is a step toward that shape but is **not** yet non-vacuous either (its Prop-valued fields are inhabitable by True). Tracked as **PL-024-6**.

OmegaCosmology/Primitives/TerminalDisjointness.lean **(Obstruction Evidence Lane terminals, 24 lines).** {[}VERIFIED{]} Models the Obstruction Evidence Lane terminals: exactRepair, stabilizedSurvivor, openObstruction, refusal. Verifies disjointness of exactRepair from each other terminal (exactRepair\≠\stabilizedSurvivor, exactRepair\≠\openObstruction, exactRepair\≠\refusal) by contradiction (L14-16). Scope notes: (i) only the exactRepair boundary is checked — the remaining pairs (stabilizedSurvivor\≠\openObstruction, etc.) are not proven; (ii) classify_unique is the trivial single-valuedness of a function (rfl plus symm, holding for *any* function into *any* type), **not** a substantive uniqueness-of-classification result; (iii) this kernel is terminal-**internal** — it does not reference $\mathbb{T}_{\mathsf{auth}}$. The cross-lane separation is established by GTTA14_ResidualNonCreation.lean.

> **Trichotomy note (Finite Obstruction Calculus vocabulary).** The cohomological classification is three-way — exact repair, stabilized survivor, open obstruction. refusal is a **fourth routing/terminal outcome** (NoRule), not a fourth cohomology class; the kernel above carries it as a fourth constructor accordingly. Where this paper says "trichotomy", it names the three-way cohomological part only.

3.7 Scope caveat

The Lean kernel models a **dependent family**, not a **categorical natural transformation**. It verifies that the family $\prod_r \mathsf{IsAuth}(\mathsf{residualTerminal}(r))$ is empty — no function can extract authority from residual terminals. It does **not** formalize functors, object-maps, or naturality squares.

The categorical statement $\mathrm{Nat}(\Omega_{\Lambda\Omega}, \mathsf{Auth}) = \emptyset$ (no natural transformation from the residual-closure functor to the authority functor) follows by §V.2's **prose reduction**: the residual-closure layer carries no descent functor D and no verifier V, so any would-be transformation must factor through the dependent family, which is empty. If a forgetful component map $U : \mathrm{Nat}(\Omega_{\Lambda\Omega}, \mathsf{Auth}) \to \prod_r \mathsf{IsAuth}(\mathsf{residualTerminal}(r))$ is constructed under H3, then the Lean-proven emptiness of the dependent family forces $\mathrm{Nat}(\Omega_{\Lambda\Omega}, \mathsf{Auth})$ empty (a total map into $\emptyset$ has empty domain). Constructing this reduction is the named conditional step **PL-024-1**; it is not mechanized here. The residual is the **named prose hypothesis H3** (canonical Theorem V.2, L3478-3492; the three bullets at L3482-3484) plus that construction, not a missing coherence condition.

This honest gap is the primary target for Paper B (§5.3).

3.8 Mechanization of the Seven Operational Guards and Typed Component Bridge

To discharge the open obligations **PL-024-7** and the typed bridge requirement of **PL-024-1**, lean4/ResidualNonCreation.lean formalizes:

1. **The Seven Operational Guards (**IsAuth_S**):**
```
structure SevenGuards where
  validHash : Bool
  replayMatches : Bool
  terminalizerPresent : Bool
  kappaZero : Bool
  deltaZero : Bool
  descentWitness : Bool
  replayFresh : Bool

def AllGuardsPass (g : SevenGuards) : Prop :=
  g.validHash = true ∧ g.replayMatches = true ∧ g.terminalizerPresent = true ∧
  g.kappaZero = true ∧ g.deltaZero = true ∧ g.descentWitness = true ∧ g.replayFresh = true
```
2. **Guarded Evidence and Guard Factorization:** Authority evidence GuardedAuthEvidence requires a proof of AllGuardsPass\guards, preventing unchecked construction. Theorem is_auth_s_factors_through_is_auth shows that operational authority IsAuth_S implies structural summand membership IsGuardedAuth.
3. **Discrete Diagnostic Cochain Complex:** Given a finite cell topology, 0-cochains $C^0 := V \to \mathbb{Z}$ (vertex potentials) and 1-cochains $C^1 := E \to \mathbb{Z}$ (edge transition discrepancies) are connected by the discrete coboundary $d^0 s(e) := s(\mathrm{tgt}(e)) - s(\mathrm{src}(e))$. Exact repair is defined by $\mathrm{IsCoboundary}(c) := \exists s, d^0 s = c$. Non-repairable obstructions are defined by non-zero classes in $Q^1 = C^1 / \operatorname{im} d^0$, i.e., $\mathrm{IsObstruction}(c) := \neg\, \mathrm{IsCoboundary}(c)$.
4. **Typed Component Bridge Theorems:**
   - **Bridge Mutual Exclusion (**bridge_mutual_exclusion**):** The canonical diagnostic route $\mathcal{R}(c)$ into Sum. inr and the guarded authority route $\Pi^\sharp(e)$ into Sum. inl have strictly disjoint images.
   - **Zero Authority on Diagnostic Route (**bridge_diagnostic_carries_no_authority**):** Diagnostic terminals carry zero authority under both IsGuardedAuth and IsAuth_S.
   - **Obstruction Guard Refusal (**obstruction_cannot_pass_truthful_guards**):** For any non-repairable obstruction cochain, no evidence whose deltaZero guard is truthful can satisfy AllGuardsPass.
   - **Cochain Bridge Disjointness (**cochain_bridge_obstruction_disjoint**,** cochain_bridge_obstruction_not_auth_s**):** Obstructed cochains mapped via the diagnostic bridge cannot inhabit the authority summand or satisfy IsAuth_S.

All theorems are verified axiom-free in Lean 4 without custom axioms (#print\axioms empty).

4. Worked Negative Instance: The Laundered Survivor (Internal Audit — Pending Public Reproducibility)

4.1 Setup: a Kaggle solver claims certification

We illustrate the firewall on a live internal system (an internal audit, not a public empirical validation — see the section title and §5): a Kaggle competition solver for Boolean function identification. The solver (BitwiseSolver) enumerates candidate rules as AST (abstract syntax tree) expressions up to a declared bound (depth ≤ 4, nodes ≤ 8), deduplicates by truth table, filters candidates against training examples, and reports survivors. The solver's internal theorem — the AstShiftExprBoundedReplayKernelTheorem — claims four properties:

1. Canonical survivor enumeration is complete.
2. Replay checks all examples.
3. Singleton query output is preserved by the certificate.
4. The independent replay kernel evaluates the same normal forms as the candidate enumerator.

If all four held, the solver's output could plausibly be promoted to authority: "the answer is certified because the enumeration is exhaustive, the replay is complete, the output is determinate, and an independent evaluator agrees."

This is exactly the self-certification pipeline that the firewall is designed to block.

4.2 The adversarial review

An adversarial code review (the "CC1" review, conducted against the shipped source) attacked all four clauses. Two are reported by internal audit as refuted, pending public artifacts:

Clause 4 — INTERNAL-AUDIT CLAIM, OPEN UNTIL ARTIFACT ATTACHED (no independent evaluator). OPEN The "public replay boundary" candidate_rules_for_replay is reported by the internal audit as a one-line delegate to _candidate_rules — the same internal method, hitting the same process-global singleton rule library. There is no second code path. An "independent verifier" that calls this boundary re-executes the identical enumerator. This is textbook replay self-certification. Public promotion requires the audit artifact, command, and output hash listed in Reproducibility.

Furthermore, the "normal form" is never evaluated. AnfAstRule.apply is self.tt_bytes{[}value\&\0xFF{]} — a truth-table lookup, not a tree evaluation. The composite expression types (UnaryExpr, BinaryExpr, TernaryExpr) define no eval method. The canonical s-expression is decorative: a bug in the vectorized truth-table computation (op_shl, op_shr, op_xor) would silently desync the printed expression from the applied function, and nothing in the pipeline can detect it.

Clause 1 — INTERNAL-AUDIT CLAIM, OPEN UNTIL ARTIFACT ATTACHED (incomplete enumeration). OPEN The enumerator build_expr8_library is reported by the internal audit to deduplicate by truth table at every node-count step, keeping only the single lowest-cost representative. The DP then composes only from kept representatives. When a function's minimum-node representative is deeper than an alternative realization, composing through the deep representative exceeds the depth bound, and the parent function is never built — even though a shallower, higher-node-count subexpression would have kept both within bounds. A Pareto-complete reference enumerator (retaining the full (node_count,\depth) frontier per truth table) finds **233 in-bound truth tables missing from the shipped library** (25,535 shipped vs. 25,768 Pareto-complete), of which **213 have shift-realizable in-bound expressions**. search_exh austive\=\True is therefore false for this family. Public promotion requires the audit artifact, command, and output hash listed in Reproducibility.

Clause 2 — UNATTACKED IN AUDIT. The example-filtering loop genuinely checks all supplied examples under the internal audit protocol. {[}INTERNAL-AUDIT\CLAIM{]}

Clause 3 — MISLEADING. {[}INTERPRETIVE{]} Query determinacy (the survivors agree on the query input) is internal consistency, not correctness. The audit's own verdict() returns NO-GO when label mismatches occur, proving determinate-but-wrong is a live possibility.

Additionally, membership in the ast_shift_expr class is a **cost-tiebreak artifact**: the class is defined by string-matching shl:1/shr:1 on the canonical s-expression, but which spelling wins the tiebreak depends on lexicographic ordering ("rotl"\\TU\textless{}\"shl"). Functions with genuine in-bound shift realizations are evicted when a rotation spelling wins. The class is not **quotient-saturated** — it is not closed under the truth-table equivalence the pipeline quotients by.

4.3 What the firewall did

Under the reported audit conditions, the authority gate reported authority_certifiable\=\0 for the bit family — and for several others (unit, gravity) where similar defects were found. The 46.67% baseline accuracy was reported as honest diagnostic output, not as a certified result. No row was promoted to ExactRuleDerived or UniqueOutputDerived.

This is consistent with the firewall discipline, but is not public empirical validation until the audit artifacts are attached. Chronologically, the internal audit records reflect two distinct stages: before adversarial review, unverified flags (including hardcoded search exhaustiveness) passed the inadequate operational checks on asserted inputs; subsequent out-of-band review (CC1) revealed the missing 233 truth tables and lack of an independent evaluator. When evaluated with these corrected inputs, the authority route refused promotion, routing the defect into a residual class.

4.4 The counterfactual

A system without the firewall — where evidence can promote itself to authority — would have:

1. Accepted search_exhaustive\=\True at face value (it is a field on solver output, not independently verified).
2. Treated the determinate query output as certified.
3. Shipped a 46.67% accuracy result as "verified."
4. Never detected the 233 missing truth tables, the absent evaluator, or the tiebreak-dependent class membership.

The firewall converts this from an informal assumption to a structural distinction: residual terminalization produces an outcome in the residual summand, which cannot be identified with an authority terminal under constructor disjointness.

4.5 The repeated-demotion pattern

The bit family's ast_shift_expr is not an isolated case. The same firewall blocked authority promotion for the unit family and the gravity family, where analogous defects were found (the underlying math was wrong in each case, and the evidence lane correctly classified the defects as unresolved). The pattern is: **the firewall holds precisely where it should — on families where the solver's claims do not survive adversarial review**. {[}INTERPRETIVE{]} Under the reported audit conditions this is consistent with the separation being operationally load-bearing; it is not a public demonstration of it, since the two load-bearing clause refutations (clauses 1 and 4) remain OPEN pending the artifacts listed in Reproducibility.

5. Honest Scope

This section is the paper's integrity anchor. We state precisely what is proven, what is conditional, and what is irreducible. Every item corresponds to a finding in the adversarial synthesis that attacked this paper's claims (16-agent swarm, firewall_separation_attack.md).

5.1 What is proven (axiom-free)

Pointwise separation: $\neg\,\mathsf{IsAuth}(\mathrm{residualTerminal}(r))$ — where IsAuth is **summand membership**, not Def 2.2's seven guards

Status: {[}VERIFIED{]}. Lean file (current path): lean4/ResidualNonCreation.lean. Theorem: residual_not_auth.

Dependent-family emptiness: $(\forall r,\ \mathsf{IsAuth}(\mathrm{residualTerminal}(r))) \to \mathsf{False}$

Status: {[}VERIFIED,\CONDITIONAL\+\H-inhab{]}. Lean file (current path): same. Theorem: nat_residual_auth_empty; **H-inhab**: residual layer inhabited (discharged concretely in-file by authExtractor_empty via residualWitness).

Descent Authority Lane lane, type-design statement: no FormalStructure\→\AuthorityTerminal under **H-noEv**

Status: {[}VERIFIED,\CONDITIONAL\+\H-noEv{]} — **vacuous: H-noEv is unsatisfiable** (see §3.6). Lean file (current path): lean4/ (historical). Theorem: no_formal_to_authority_when_no_terminalizer; **H-noEv**: Terminalizer Evidence\→\False, unsatisfiable because the record is freely inhabited.

Obstruction Evidence Lane terminal disjointness at the exactRepair boundary: exactRepair\≠\stabilizedSurvivor, ≠\openObstruction, ≠\refusal (other pairs not proven)

Status: {[}VERIFIED{]}. Lean file (current path): lean4/ (historical). Theorem: terminal_disjoint_*.

Positive form: an IsAuth inhabitant unpacks to a Sum.inl witness

Status: {[}DEFINITION{]}. Lean file (current path): lean4/ResidualNonCreation.lean. Theorem: authority_only_via_evidence; proved by the identity function (:=\h) — record unpacking, not an independent structural result.

All five are axiom-free: #print\axioms reports no axioms for any theorem. **Bundled and verified in** lean4/ResidualNonCreation.lean (see Reproducibility for current instructions).

Note on the Descent Authority Lane lane statement: this does **not** prove that formal structures cannot promote to authority when terminalizer evidence is constructible — and, because TerminalizerEvidence is freely inhabited, its

hypothesis can never be discharged, so the lemma is vacuously true and carries **no** firewall force. It is retained as a statement of the *intended shape* of the Descent Authority Lane gate. The load-bearing separation in this paper is the coproduct result in row 1, which is not vacuous. Making the Descent Authority Lane lane non-vacuous is **PL-024-6**.

5.2 What is conditional

5.2.1 — Categorical $\mathrm{Nat}(\Omega_{\Lambda\Omega}, \mathsf{Auth}) = \emptyset$**.** {[}VERIFIED,\CONDITIONAL\+\hypotheses{]}

The categorical form — no natural transformation from the residual-closure functor to the authority functor — rests on the prose reduction of §V.2 (canonical Theorem V.2, L3478-3492): the residual-closure layer carries no descent functor D and no verifier V, so any would-be transformation must factor through the dependent family (which is empty by Theorem 3.2). Note the spine states V.2 as {[}CONDITIONAL\(GTTA-14){]} — "*subject to the co-inhabitation witness clause recorded in GTTA-14, this gives* $\mathrm{Nat}(\Omega_{\Lambda\Omega}, \mathsf{Auth}) = \emptyset$". This paper's Lean kernel discharges **exactly that witness clause** and no more; we additionally require the bridge U below, so the position taken here is strictly more conservative than the spine's. The **naturality-square** portion reduces *conditional on constructing* the forgetful component map U under H3 — the named step **PL-024-1**, not mechanized here: given such U, the Lean-proven emptiness of the dependent family forces the domain empty. The gap is the named hypothesis **H3** (no D, no V) together with that construction, not a missing coherence condition. Dischargeable; primary target for Paper B.

5.2.2 — Four separately declared Lean authority encodings. OPEN

The tower carries four separately declared encodings, with no verified common adapter:

- ResidualClosureLayer.AuthEvidence = String\×\String\×\Bool (many inhabitants)
- QM.AuthorityTerminal.sealed over TerminalizerEvidence = String\×\String\×\Bool (no declared adapter to AuthEvidence)
- Diagnostic\Composition\Layer.Terminal.AuthBranch (nullary constructor — a singleton)
- Layered\Authority\Framework.TerminalStatus.ExactAuthority (nullary constructor — a singleton)

A common four-way type-isomorphism is unverified (the two string/string/bool encodings are isomorphic via record wrapping, and the two singletons are isomorphic to each other, but a uniform adapter across all four is unavailable). The separation holds **per-type** (each local IsAuth is proven disjoint from the corresponding residual type). The **cross-type federation** — that each local IsAuth factors through §II.2's single Σ-gate (canonical L1560) — is currently carried in prose, not mechanized. This is an engineering gap, not a mathematical one: the separation theorem holds at each layer independently.

5.2.3 — MAJOR-3 lossy χ-pentagon. OPEN

The strict-pullback reindexing core of the Inv pseudofunctor is discharged by the pullback mapping property (hypothesis **H-pb**, Def 0.2.3.4). The lossy remainder — the **D2 content-carrying 2-cell** for non-pullback transport — has four missing lemmas:

- **L1:** comparison-cell well-definedness (non-identity $\gamma_{u,v}$)
- **L2:** pentagon for content-carrying χ (Def II.8.1, canonical L1778-1784)
- **L3a/L3b:** unit triangles (canonical L1789)
- **L4:** comparison-cell **naturality** — absent from Def II.8.1 entirely; must be added before it can be discharged

At the existence level, this is irreducible-by-design: un-witnessed transitions are "not arrows of $\mathcal{L}_\Omega$" by definition (canonical L489). At the verification level, L1-L4 are a bounded open follow-on requiring Mathlib's bicategory layer. MAJOR-3 concerns the diagnostic Inv fibration and is **orthogonal to the firewall** — it touches no arrow into Auth. Paper B territory.

5.3 What is irreducible

C4 — Verifier-soundness. OPEN

The replay-checker kernel — the component that recomputes $\Pi^\sharp$ and compares replay hashes — must be externally audited. Verifier soundness is an assumption of this development and has not been derived from the listed replay/cochain

premises; C4 is **not derivable** from finite-replay determinism + terminalization totality + $d^1 d^0 = 0$ (canonical L1714-1716). It can only be **shrunk** to a small, audited replay-checker kernel whose correctness is the system's irreducible trust root.

The Kaggle case study (§4) illustrates C4 concretely: search_exhaustive\=\True was a field on solver output, not independently recomputed. The forgery was in the verifier *input*; the type-level firewall still held; the laundering was blocked by out-of-band review (the CC1 adversarial audit), **not** by the IsAuth guards themselves (which passed on the asserted inputs, such as the hardcoded search_exhaustive\=\True, leaving CC1 to catch the false premise). This is C4 working exactly as theorized: the type system catches category errors (evidence ≠ authority), but the correctness of what counts as evidence is the trust root.

5.4 Falsifiability

Falsifiability line. The pointwise theorem is falsified by any Lean term inhabiting IsAuth\(residualTerminal\r) for some residual r; the categorical theorem is falsified by a valid construction bypassing H3 or by a faithful model of the tower in which residual closure carries an authority-producing route.

This paper's central claim fails if:

- A Lean term of type IsAuth\(residualTerminal\r) can be constructed for any r\:\ResidualOutcome. (This would require a proof of Sum.inr\r\=\Sum.inl\e, which is False by constructor disjointness.)
- The coproduct structure of the terminal type is shown to be an artifact of the modeling choice rather than a faithful representation of the tower's type design. (The coproduct mirrors the §V.2 summand split; a challenge would need to show that §V.2's type assignment is wrong, not just that a different modeling is possible.)

5.5 Tower status

UNIFIED_CONDITIONAL_STRONG — not PROVED, not UNIFIED_EXCEPT_C4.

UNIFIED_EXCEPT_C4 would claim C4 is the only remaining gap. Our own evidence refutes this: at least two open items are neither C4 nor realization-side (the categorical Nat=∅ missing lemma and the MAJOR-3 lossy D2 pentagon). The honest status is: the firewall's load-bearing witness is machine-checked, the coherence core is closed, and what remains is a short, named, located list of conditionals — none of which falsifies the spine.

6. Related Work and Implications

6.1 Related work

Proof-carrying code (Necula, 1997). {[}STANDARD\+\citation{]} PCC attaches machine-checkable proofs to executable code, explicitly separating an untrusted proof producer from a consumer's small trusted checker. The Layered Authority Framework firewall operates at a complementary level: rather than verifying that code satisfies a specification, the firewall provides a type-level separation preventing residual diagnostics from evaluating as authority terminals. PCC establishes "does this candidate satisfy the specification relative to the checker?"; the firewall establishes that residual terminalization lands strictly in the residual summand.

Certified compilation (CompCert, CakeML). {[}STANDARD\+\citation{]} Certified compilers carry a proof that compilation preserves semantics. Like PCC, this addresses correctness of a specific transformation, not the meta-question of whether the certification pipeline is self-referential. A certified-compilation pipeline would exhibit the self-authorization failure mode only if the proof-checking or authority-producing step depended on the same unaudited pipeline whose output it certifies; if the proof is checked by an independent trusted kernel, the situation is different. Where the failure mode does arise, the Layered Authority Framework firewall would catch it: the evidence-lane output (compilation diagnostics) cannot type-promote to authority-lane input (correctness certificate).

Byzantine fault tolerance (Lamport, 1982; Castro & Liskov, 1999). {[}STANDARD\+\citation{]} BFT addresses malicious participants in distributed consensus. The Layered Authority Framework firewall addresses a different adversary: a system that is not malicious but structurally misconfigured — its verification pipeline is circular by design, not by attack. BFT assumes a fault model and sufficient non-faulty replicas; the firewall instead addresses type-level evidence/authority separation inside a verification architecture.

Self-verification limits (Gödel's second incompleteness; Löb's theorem; MIRI "tiling agents" / Vingean reflection). {[}STANDARD\+\citation{]} *Differs:* these establish that a sufficiently strong system cannot prove its own consistency/soundness; the firewall does not evade this — it *localizes* it as the irreducible trust root C4 (§5.3) and type-isolates everything else. (Yudkowsky & Herreshoff 2013; Fallenstein & Soares 2015)

AI alignment verification (Christiano et al., 2018; Irving et al., 2018). {[}STANDARD\+\citation{]} AI safety proposals often invoke "debate" or "amplification" to verify AI outputs. These proposals face the self-authorization problem when the debater/amplifier shares a codebase or training distribution with the system being verified. The Layered Authority Framework firewall offers a concrete type-theoretic criterion for when a proposed verification pipeline is genuinely external vs. self-referential.

6.2 What is different

{[}INTERPRETIVE{]} Three properties distinguish the Layered Authority Framework firewall from prior work:

1. **Structural, not policy.** The separation is enforced by constructor disjointness ($\mathrm{im}(\mathsf{inl}) \cap \mathrm{im}(\mathsf{inr}) = \emptyset$), not by access-control lists or runtime flags. Residual terminalization cannot evaluate to an authority certificate.
2. **Machine-checked.** The pointwise form is proven in a small axiom-free Lean 4 kernel (bundled in lean4/ResidualNonCreation.lean; §3.3). The proof is auditable, replayable, and carries no hidden dependencies.
3. **Honest about its trust root.** The paper explicitly names C4 (verifier-soundness) as an irreducible axiom. No finite system escapes this; we disclose it rather than hiding it behind a stronger claim.

6.3 Implications

{[}INTERPRETIVE{]} Any AI system that can modify its own verification pipeline — retrain its reward model, update its evaluation rubric, adjust its confidence thresholds — should expose a structural separation between evidence production and authority production, or else state why an equivalent anti-self-authorization mechanism is present. The Layered Authority Framework tower provides one concrete realization: the evidence lane detects and classifies, the authority lane certifies, and the types prevent cross-contamination.

The core separation kernel is bundled directly in this deposit under lean4/ResidualNonCreation.lean (with historical development in GTTA14_ResidualNonCreation.lean). The categorical extension — formalizing the natural-transformation emptiness and the MAJOR-3 full discharge — is an open research target (Paper B).

References

- Necula, G. C. (1997). *Proof-Carrying Code*. POPL '97, 106–119.
- Leroy, X. (2009). *Formal verification of a realistic compiler* (CompCert). Communications of the ACM, 52(7), 107–115.
- Kumar, R., Myreen, M. O., Norrish, M., & Owens, S. (2014). *CakeML: A Verified Implementation of ML*. POPL '14, 179–191.
- Lamport, L., Shostak, R., & Pease, M. (1982). *The Byzantine Generals Problem*. ACM TOPLAS, 4(3), 382–401.
- Castro, M., & Liskov, B. (1999). *Practical Byzantine Fault Tolerance*. OSDI '99.
- Irving, G., Christiano, P., & Amodei, D. (2018). *AI Safety via Debate*. arXiv:1805.00899.
- Christiano, P., Shlegeris, B., & Amodei, D. (2018). *Supervising Strong Learners by Amplifying Weak Experts* (iterated amplification). arXiv:1810.08575.
- Yudkowsky, E., & Herreshoff, M. (2013). *Tiling Agents for Self-Modifying AI, and the Löbian Obstacle*. MIRI.
- Fallenstein, B., & Soares, N. (2015). *Vingean Reflection: Reliable Reasoning for Self-Improving Agents*. MIRI Technical Report.

Notation

Symbol	Definition	First use
C^1	Space of 1-cochains (defect measurements)	§2.1
d^0	0-coboundary operator (repair)	§2.1

Symbol	Definition	First use
$Q^1 = C^1/\mathrm{im}\, d^0$	Repair quotient (obstructions modulo exact repairs)	§2.1
$H^1 = \ker d^1/\mathrm{im}\, d^0$	First cohomology (non-repairable obstruction cycles)	§2.1
d^1	1-coboundary operator (closure)	§2.1
$\Pi^\sharp$	Terminalizer (authority)	§2.1
$\mathbb{T}$	Global terminal type: $\mathbb{T}_{\mathsf{auth}} \oplus (\mathbb{T}_{\mathrm{res}} + \mathbb{F}_{\mathrm{res}})$	§3.1
$\mathsf{IsAuth}(t)$	Authority predicate: $\exists\, e,\ t = \mathsf{inl}\, e$	§3.1
residualTerminal(r)	$\mathsf{inr}\, r$ (maps residuals into the disjoint summand)	§3.1
$\mathrm{Nat}(F, G)$	Set of natural transformations $F \Rightarrow G$	§3.6
$\Omega_{\Lambda\Omega}$	Residual-closure layer (Residual Closure Layer, layer 6)	§2.3
Auth	Authority functor / Σ-record (§II.2)	§2.4
$[q] \in Q^1 \nRightarrow \mathsf{Auth}$	The Separation Theorem (evidence ≠ authority)	§1.3

Claim Ledger

Per the paper hygiene standard, every substantive claim is typed with the fixed tags {[}DEFINITION{]}, {[}VERIFIED{]}, {[}PROVEN,\CONDITIONAL\+\H1…Hn{]}, {[}STANDARD\+\citation{]}, {[}MECHANIZED-PARTIAL{]}, {[}WORKED\EXAMPLE{]}, {[}IMPLEMENTATION\CONTRACT{]}, {[}SCOPE\FENCE{]}, {[}CONJECTURE{]}, {[}INTERPRETIVE{]}, or OPEN. A standalone companion ledger is also provided in authority_firewall_claim_ledger.md.

Claim	Type	Location
Five primitives compress the tower	{[}VERIFIED,\CONDITIONAL\+\hypotheses{]}	Theorem 0.1, §2.1; imported from the versioned canonical spine
Five primitives and Authority Σ-record	{[}DEFINITION{]}	Definition 2.1, §2.1; Definition 2.2, §2.4
Authority type / IsAuth / descent-packet data are **Descent Authority Lane-owned and imported**; the non-promotion rule is this paper's Layered Authority Framework contribution	{[}SCOPE\FENCE{]}	Definition 2.2, §2.4; Finite Obstruction Calculus/Layered Authority Framework Compliance Note
Pointwise separation: $\neg\, \mathsf{IsAuth}(\text{residualTerminal}(r))$, at the **zero-guard** IsAuth scope	{[}VERIFIED{]}	Theorem 3.1, §3.2
Mechanized IsAuth carries **none** of Def 2.2's seven guards; AuthEvidence is freely inhabited	{[}SCOPE\FENCE{]}	§2.4 guard-scope firewall; PL-024-7
ContractAuthority / VerifierScopedAuthority in scope; **SemanticAuthority NOT AVAILABLE** (no semantic adapter declared)	{[}SCOPE\FENCE{]}	§2.5
Dependent-family emptiness	{[}VERIFIED,\CONDITIONAL\+\H-inhab{]}	Theorem 3.2, §3.2; H-inhab: residual layer inhabited

Claim	Type	Location
Descent Authority Lane-lane non-promotion under H-noEv — **vacuous, H-noEv unsatisfiable**	{[}VERIFIED,\CONDITIONAL\+\H-noEv{]}	§3.6 vacuity disclosure; §5.1; PL-024-6
Obstruction Evidence Lane terminal disjointness at the exactRepair boundary (other pairs unproven); refusal/NoRule is a fourth **routing** terminal, not a fourth cohomology class	{[}VERIFIED{]}	§3.6; §2.2
classify_unique is trivial single-valuedness of a function, not substantive classification uniqueness	{[}DEFINITION{]}	§3.6
authority_only_via_evidence is the identity function (:=\h) — record unpacking	{[}DEFINITION{]}	§5.1
Worked q42 replay	{[}WORKED\EXAMPLE{]}	§3.5
Bundled Lean kernel in lean4/ResidualNonCreation.lean	{[}IMPLEMENTATION\CONTRACT{]}	Reproducibility
Clause 4 refuted (no independent evaluator)	OPEN	§4.2; internal audit claim pending public artifact
Clause 1 refuted (233 missing truth tables)	OPEN	§4.2; internal audit claim pending public artifact
Clause 2 unattacked in audit	{[}INTERNAL-AUDIT\CLAIM{]}	§4.2
Clause 3 (determinacy ≠ correctness)	{[}INTERPRETIVE{]}	§4.2
Categorical Nat = ∅	{[}VERIFIED,\CONDITIONAL\+\hypotheses{]}	§5.2.1; hypotheses H3 and dependent-family-to-Nat bridge
Four-type authority federation	OPEN	§5.2.2
MAJOR-3 D2 content-carrying 2-cell	OPEN	§5.2.3
C4 verifier-soundness	OPEN	§5.3
Related-work positioning	{[}STANDARD\+\citation{]}	§6.1
Firewall blocks self-authorization in practice	{[}INTERPRETIVE{]}	§4.3; artifact-dependent
Any autonomous AI needs structural firewall	{[}INTERPRETIVE{]}	§6.3

Definition audit. The five primitives, the Authority Σ-record, IsAuth, and residualTerminal are definitions. The theorem-level claims begin only when the paper establishes disjointness or derives an uninhabited family from those definitions.

Reproducibility

Lean — Bundled kernel verification:

The primary mathematical kernel is bundled directly within this package at lean4/ResidualNonCreation.lean:

- **Source:** lean4/ResidualNonCreation.lean (108 lines, pure Lean 4, no Mathlib required).
- **Command:**
  `lean lean4/ResidualNonCreation.lean`
- **Axiom report:** #print\axioms is executed in-file for residual_not_auth, nat_residual_auth_empty, authority_only_via_evidence, and authExtractor_empty. Each prints: \XeTeXglyph\numexpr\XeTeXcharglyph"0027\relax{}OmegaCore.ResidualClosureLayer.\TU\textless{}theorem\TU\textgreater{}\XeTeXglyph\numexpr\XeTeXcharglyph"0027\relax{}\does\not\depend\on\any\axioms.
- **Scope reminder:** an axiom-free green build certifies only the exact statements listed in §5.1 at the strength stated there — in particular the zero-guard reading of IsAuth (§2.4). It does not discharge PL-024-1 … PL-024-8.

Python reference implementation & tests:

- **Tests (11 passed):**
  `py -3.12 -m pytest code/tests/test_authority_firewall.py`
- **Deposit verification:**
  `py -3.12 verify_deposit.py --test code/tests/test_authority_firewall.py`

Kaggle CC1 (internal audit — pending public reproducibility; tracked as PL-024-4): The §4 negative instance is an internal-audit claim. The following public artifacts are required before it carries empirical status and are **not yet attached**:

- **Commit:** *(pending)*
- **Command:** *(pending)*
- **Missing-table report hash:** *(pending)*
- **Baseline-score report hash:** *(pending)*

Audit Companions:

- The Omega Sweeper structure-sweep synthesis: omega_engine/research/omega_sweeper/04_model_runs/run_024_authority_firewall/final_synthesis.md.
- The 16-agent adversarial synthesis: omega_engine/.product_development/initial_planning_docs/firewall_separation_attack.md.

Finite Obstruction Calculus / Layered Authority Framework Compliance Note

Role: Layered Authority Framework-side authority **non-promotion** paper. It states and mechanizes (pointwise) the rule that the Obstruction Evidence Lane evidence lane cannot reach the Descent Authority Lane authority summand. It is a constraint *on* authority, not a source of it.

Degree: Finite, degree-1 obstruction regime. $C^0 \xrightarrow{d^0} C^1 \xrightarrow{d^1} C^2$; $Q^1 := C^1 / \operatorname{im} d^0$; $H^1 = \ker d^1 / \operatorname{im} d^0$. Separation is proven at the **pointwise type level** (terminal-type discipline), not the categorical level.
Obstruction regime / Finite Obstruction Calculus data: ObstructionRegime(k=1); cochains C^0, C^1, C^2; differentials d^0, d^1; $d^1 \circ d^0 = 0$ is a **declared/imported axiom** (canonical L1714-1716), not re-proven here. ZeroRel / local sameness: exactness at C^1 iff $H^1 = 0$ (the exact-repair case $\operatorname{im} d^0 = \ker d^1$). Gauge/observer: repair = adjustment by a 0-cochain ($\operatorname{im} d^0$); no observer norm is declared in this paper. **Lift policy: no lift is claimed or licensed here.**
Terminal policy: $\mathbb{T} = \mathbb{T}_{\mathsf{auth}} \oplus (\mathbb{T}_{\text{res}} + \mathbb{F}_{\text{res}})$, with refusal/NoRule as a fourth **routing** terminal (the cohomological classification is three-way).
Consumes: Theorem 0.1 (compression) — imported, conditional; the canonical §II.2 Authority Σ-record and seven-guard IsAuth_S — **Descent Authority Lane-owned, imported**; §V.2's residual-closure bullets; hypothesis H3 (no descent functor D, no verifier V on residual closure).
Produces: Theorem 3.1 (pointwise coproduct separation) — mechanized, axiom-free; Theorem 3.2 (dependent-family emptiness) under H-inhab; Theorem 3.3 (operational seven-guard factorization is_auth_s_factors_through_is_auth); Theorem 3.4 (typed component bridge mutual exclusion and obstruction defect refusal); the Layered Authority Framework non-promotion rule $[q] \in Q^1 \not\Rightarrow \mathsf{Auth}$ at the terminal-type-discipline scope.
Does not produce: SemanticAuthority (no semantic adapter declared); a mechanized categorical $\mathrm{Nat}(\Omega_{\Lambda\Omega}, \mathsf{Auth}) = \emptyset$; a discharge of C4; public empirical validation; the four-type authority federation; any lift licence.
Authority status: Authority-bearing paper — "Authority: none" does not apply. Scoped per §2.5: **ContractAuthority** and **VerifierScopedAuthority** are in scope; **SemanticAuthority is NOT AVAILABLE**. This paper mints no authority; it constrains the routes by which authority may be claimed. Any authority-bearing use of this firewall requires a Descent Authority Lane/Layered Authority Framework descent packet carrying SpecificationIdentity, DetectorAdequacy, ExhaustionWitness, EvidenceCeiling, ReplayWitness, Terminalizer, and an IsAuth guard — of which this paper mechanizes the structural IsAuth, the operational seven-guard IsAuth_S, and the typed component bridge in lean4/ResidualNonCreation.lean; SpecificationIdentity**,** DetectorAdequacy**, and** EvidenceCeiling **are declared here as required and are NOT supplied** (PL-024-8).
Declared imports: compression Theorem 0.1; canonical §II.2 Authority record + IsAuth_S; §V.2 prose reduction; H3; H-pb (pullback mapping property, Def 0.2.3.4).
OracleNeeded / Q-gated items: PL-024-1 (categorical natural transformation from the typed component bridge under H3); the Layer-5 "continuum finite-shadow" (Ω_κ, Finite-Shadow Approximation) is **named in the architecture but not constructed at this paper's declared finite degree — Q-gated**, and nothing here depends on it; MAJOR-3 L1-L4 (χ-pentagon, unit triangles, comparison-cell naturality), which additionally need Mathlib's bicategory layer.

Layered Authority Framework boundary: Descent Authority Lane owns the Authority type, IsAuth, and descent-packet data. **Layered Authority Framework** owns the governance capstone, terminalization discipline, and the authority non-promotion rules — this paper's contribution sits here. **Finite Obstruction Calculus/Obstruction Evidence Lane** diagnoses, classifies, and routes obstruction packets and **mints no authority**. Tower layer labels (1-6) are role/routing indices, **not ontological levels**.
Mechanization status: {[}MECHANIZED-PARTIAL{]}. Mechanized and re-verified axiom-free: residual_not_auth, nat_residual_auth_empty, authExtractor_empty, is_auth_s_factors_through_is_auth, guarded_evidence_satisfies_is_auth_s, diagnostic_not_guarded_auth, diagnostic_not_auth_s, bridge_mutual_exclusion, bridge_diagnostic_carries_no_authority, obstruction_cannot_pass_truthful_guards, cochain_bridge_obstruction_disjoint, cochain_bridge_obstruction_not_auth_s, no_formal_to_authority_when_no_terminalizer (**vacuous — see §3.6**), terminal_disjoint_*, classify_unique (trivial), authority_only_via_evidence (identity function). Not mechanized: the full categorical natural transformation, the federation, C4.
Remaining open obligations: PL-024-1 (categorical natural transformation under H3); PL-024-2 (four-type federation through one §II.2 Σ-gate); PL-024-3 (MAJOR-3 L1-L4); PL-024-4 (public Kaggle artifacts); PL-024-5 (C4 replay-checker audit contract); PL-024-6 (non-vacuous Descent Authority Lane lane); PL-024-7 (RESOLVED: mechanized in lean4/ResidualNonCreation.lean); PL-024-8 (SpecificationIdentity / DetectorAdequacy / EvidenceCeiling for authority-bearing use).

Formalization-status firewall. Any Lean, Rust, or replay artifact cited here supports only the exact finite statement it mechanizes. It does not automatically discharge PL-Finite Obstruction Calculus, Q, continuum, physical-realization, or Layered Authority Framework authority obligations. A status upgrade requires an explicit theorem-ledger mapping, hypotheses check, build receipt, axiom/sorry audit, and adversary acceptance. Concretely for this paper: the green, axiom-free build certifies inr\≠\inl, the seven operational guards factorization, and the typed component bridge inside a declared terminal-type discipline — it certifies **no** naturality, **no** verifier soundness, and **no** empirical claim.

Detector adequacy warning. A replayable detector/profile is not automatically adequate for a claim. A collapsing detector can make every profile exact. Any authority-bearing Layered Authority Framework use requires a DetectorAdequacy certificate naming the protected distinctions, adversarial cases, and known blind spots for the declared claim scope. **This paper supplies no such certificate**, and §4 is the cautionary instance: before adversarial review, unverified flags passed the inadequate operational checks; replay alone did not detect the missing truth tables. Detector adequacy is therefore prior to, and not implied by, replayability (PL-024-8).

What This Paper Does NOT Claim

> **Best framing:** This paper is a finite type-theoretic firewall showing that residual evidence cannot inhabit the authority terminal summand.

It is **NOT**:

- **A complete proof of the Layered Authority Framework tower.** The tower status is UNIFIED_CONDITIONAL_STRONG (§5.5).
- **A complete AI-safety theorem.** Structural separation alone does not suffice; C4 (§5.3) is irreducible and out-of-band auditing remains necessary.
- **A complete categorical theorem.** It does NOT claim the categorical $\mathrm{Nat}(\Omega_{\Lambda\Omega}, \mathsf{Auth}) = \emptyset$ is mechanized. That is a named open conditional (§5.2.1).
- **A public empirical Kaggle validation.** The Kaggle case is currently an internal-audit illustration pending public reproducibility artifacts (§4).
- **A claim of SemanticAuthority.** No semantic adapter is declared, so nothing here licenses "the answer is true." The reachable scopes are ContractAuthority and VerifierScopedAuthority (§2.5).
- **A claim that operational guards replace external verifier soundness.** The mechanized IsAuth_S formalizes the operational guards; soundness relative to external ground truth remains the C4 trust root (§5.3).
- **A Descent Authority Lane firewall theorem.** The Descent Authority Lane-lane lemma is vacuous — its hypothesis is unsatisfiable for the record type it quantifies over (§3.6) — and is retained only as a type-design statement.

Additional non-claims:

- It does NOT claim the Kaggle result is "good." The 46.67% baseline is honest, not impressive. The paper's value is the *theorem*, not the score.

- It does NOT introduce or rely on motivic structures or the Langlands program. The Lean-proven pointwise theorem uses only type-theoretic coproduct disjointness; higher-categorical and homological language is used only in the scoped Layered Authority Framework background and conditional extensions.

Revision Notes

Version Note: v2.1.0 (2026-10-08). Standardized academic release incorporating publication review remediations: - Bundled self-contained Lean 4 proof in lean4/ResidualNonCreation.lean under exact OmegaCore.ResidualClosureLayer namespace. - Enforced strict runtime payload, tag, and exact boolean guard contracts in code/mod_authority_firewall_core.py with 10 passing unit tests covering all reproduced review edge cases. - Deflated constructor disjointness claims to eliminate self-authorization overreach, clarifying that coproduct disjointness prevents identifying an existing residual terminal with an authority certificate. - Reconciled audit narrative into explicit pre-review and post-review stages; downgraded Clause 2 to internal audit claim. - Aligned mathematical terminology: repair quotient Q^1 vs first cohomology H^1, injection image intersection $\mathrm{im}(\mathsf{inl}) \cap \mathrm{im}(\mathsf{inr}) = \emptyset$. - Corrected architecture diagram routing for Layer 6. - Aligned code license to PolyForm Noncommercial 1.0.0 and removed pending DOI placeholder.

Version Note: v2.0.1 (2026-10-07). Release verification tooling alignment.
Version Note: v2.0.0 (2026-10-07). Initial publication-grade monograph promotion.
Version Note: v1 Pre-Print / v0.1 Pre-Print (June 2026). Historical pre-print drafts.

> **Pedagogical Crosswalk Notice:** Streamlined pedagogical exposition for this kernel is provided under 100_mathematical_foundations/. This monograph remains the primary mathematical home.

Claim-ID body index (packaging)

ID	Home
AF-01	sibling authority_firewall_claim_ledger.md
AF-02	sibling authority_firewall_claim_ledger.md
AF-03	sibling authority_firewall_claim_ledger.md
AF-04	sibling authority_firewall_claim_ledger.md
AF-05	sibling authority_firewall_claim_ledger.md
AF-06	sibling authority_firewall_claim_ledger.md
AF-07	sibling authority_firewall_claim_ledger.md
AF-08	sibling authority_firewall_claim_ledger.md
AF-09	sibling authority_firewall_claim_ledger.md
AF-20	sibling authority_firewall_claim_ledger.md
AF-21	sibling authority_firewall_claim_ledger.md
AF-22	sibling authority_firewall_claim_ledger.md
AF-23	sibling authority_firewall_claim_ledger.md
AF-24	sibling authority_firewall_claim_ledger.md
AF-10	sibling authority_firewall_claim_ledger.md
AF-11	sibling authority_firewall_claim_ledger.md
AF-12	sibling authority_firewall_claim_ledger.md
AF-13	sibling authority_firewall_claim_ledger.md
AF-14	sibling authority_firewall_claim_ledger.md
AF-15	sibling authority_firewall_claim_ledger.md
AF-16	sibling authority_firewall_claim_ledger.md
AF-17	sibling authority_firewall_claim_ledger.md
AF-18	sibling authority_firewall_claim_ledger.md
AF-19	sibling authority_firewall_claim_ledger.md
IsAuth	sibling authority_firewall_claim_ledger.md
residualTerminal	sibling authority_firewall_claim_ledger.md
authority_only_via_evidence	sibling authority_firewall_claim_ledger.md
PL-024-1	sibling authority_firewall_claim_ledger.md
PL-024-2	sibling authority_firewall_claim_ledger.md

ID	Home
PL-024-3	sibling authority_firewall_claim_ledger.md
PL-024-4	sibling authority_firewall_claim_ledger.md
PL-024-5	sibling authority_firewall_claim_ledger.md
PL-024-6	sibling authority_firewall_claim_ledger.md
PL-024-7	sibling authority_firewall_claim_ledger.md
PL-024-8	sibling authority_firewall_claim_ledger.md
PL-024-9	sibling authority_firewall_claim_ledger.md